

Contents

1 The Gram-Schmidt Process *3*

1.1 The Process *3*

1.2 Example Calculation *4*

1.3 The Gram-Schmidt Process in Python *6*

1.4 Where to Learn More *7*

2 Singular Value Decomposition *9*

2.1 Definition *9*

2.2 Applications of SVD *9*

2.3 Calculating SVD Manually *10*

2.4 Singular Value Decomposition with Python *13*

2.5 Sign Ambiguity *14*

2.6 SVD Applied to Image Compression *14*

2.7 Where to Learn More *15*

3 Tackling Difficult Problems: Positive Semidefinite Matrices *17*

3.1 NP Problems *17*

3.2 A Past Approach: Minimizing Errors in Neural Networks *18*

3.3 Positive Definite and Semidefinite Matrices *19*

3.4 Identifying and Constructing a Positive Semidefinite Matrix *20*

3.5 The Max Cut Problem *21*

3.6 The Max Cut Problem Solved in Python *23*

4 Introduction to C *25*

4.1 Compiled vs. Interpreted 25
4.2 Your First C Program 26
4.3 Types and Variables 26
4.4 Printing with `printf` 27
4.5 Conditionals and Loops 28
4.5.1 if / else 28
4.5.2 for and while loops 29
4.6 Functions 29
4.7 The Stack: Blackboards for Functions 30
4.7.1 Parameters are copies 31
4.8 Addresses and Pointers 31
4.8.1 Reading and writing through a pointer 32
4.8.2 NULL pointers 33
4.9 Pass-by-Reference 33
4.10 Structs: Grouping Related Data 35
4.11 The Heap 36
4.11.1 The arrow operator 36
4.11.2 Releasing heap memory 37
4.11.3 A complete heap example 37
4.11.4 Heap arrays 38
4.12 What Python Objects Actually Are 38
4.13 Exercises 39

5 Linked Lists and C++ transition ***41***
5.1 From C to C++ 41
5.2 Linked Lists 41
5.3 Doubly Linked Lists 46

6 Bitmaps ***49***
6.1 Bitmap Structure 50
6.1.1 Bitmap Headers 50
6.1.2 Padding 51
6.2 Creating a Bitmap with C++ 57
6.3 Connection to transformations 63
6.4 Summary 64

A Answers to Exercises ***67***

Index 73

Chapter 1

The Gram-Schmidt Process

The Gram-Schmidt Process is a method use to transform a set of linearly independent vectors to a set of orthogonal (perpendicular) vectors. The original vectors and the transformed vectors span the same subspace.

The process was named after two mathematicians: Jørgen Pedersen Gram, a Danish actuary mathematician, and Erhard Schmidt, a German mathematician. The men developed the orthogonalization process independently. Gram introduced the process in 1883, whereas Schmidt did his work in 1907. It was not named the Gram-Schmidt process until sometime later, after both mathematicians became well known in the mathematical community.

In the last chapter, you learned how to calculate a projection of one vector on another. Given two vectors, u and v, the projection separates u into the part that is orthogonal to v and the part that has a dependency with v. The Gram-Schmidt Process builds on the notion of projections to iteratively strip away dependencies between vectors until the vectors that remain are orthogonal. If there happens to be a vector that is a linear combination of the others, that vector will be reduced to the zero vector. The vectors that remain define a new basis for space spanned by the original set of vectors.

Gram-Schmidt has many practical applications in science and engineering, such as:

1. In signal processing, it can represent an audio signal with fewer components making it easier to isolate and remove noise.
2. In statistics and data analysis, it can reduce the complexity of a dataset so that it is easier to see which aspects or features contribute to the analysis.

1.1 The Process

The Gram-Schmidt Process orthonormalizes a set of vectors in an inner product space, most commonly the Euclidean space $\mathbb{R}^n$. The process takes a finite, linearly independent set $S = \{v_1, v_2, \ldots, v_k\}$ for $k \leq n$, and generates an orthogonal set $S' = \{u_1, u_2, \ldots, u_k\}$ that spans the same k-dimensional subspace of $\mathbb{R}^n$ as S.

Let's look at how the process works. Given a set of vectors $S = \{v_1, v_2, \ldots, v_k\}$, the Gram-Schmidt Process is as follows:

1. Let $u_1 = v_1$.
2. For $j = 2, 3, \ldots, k$:
   (a) Let $w_j = v_j - \sum_{i=1}^{j-1} \frac{\langle v_j, u_i \rangle}{\langle u_i, u_i \rangle} u_i$
   (b) Let $u_j = w_j$

Here, $\langle ., . \rangle$ denotes the inner product.

The set of vectors $S' = \{u_1, u_2, \ldots, u_k\}$ obtained from this process is orthogonal, but not necessarily orthonormal. To create an orthonormal set, you simply need to normalize each vector u_i to unit length. That is, $u_i' = \frac{u_i}{\|u_i\|}$, where $\|.\|$ denotes the norm (or length) of a vector.

Among other things, making vectors orthonormal simplifies calculations makes it easier to define rotations and transformations, and provides a framework for calculations in fields such as quantum mechanics.

1.2 Example Calculation

Given a set of linearly independent vectors, we will use the Gram-Schmidt Process to find an orthogonal basis.

Let

$$W = Span(x_1, x_2, x_3)$$

where

$$x_1 = (1, 2, -2)$$
$$x_2 = (1, 0, -4)$$
$$x_3 = (5, 2, 0)$$

The three orthogonal vectors will define the same subspace as the original vectors.

The first vector of the orthogonal subspace is easy to define. We set it to be the same as x_1.

$$v_1 = x_1 = (1, 2, -2)$$

The second orthogonal vector is a projection of x_2 onto v_1. You learned projections in the last chapter, so this should be fairly straightforward.

$$v_2 = x_2 - \frac{x_2 v_1}{v_1 v_1} v_1$$

Substitute the values:

$$v_2 = (1, 0, -4) - \frac{(1, 0, -4)(1, 2, -2)}{(1, 2, -2)(1, 2, -2)} (1, 2, -2)$$

Calculate the coefficient for v_1:

$$v_2 = (1, 0, -4) - \frac{9}{9}(1, 2, -2)$$

Perform the subtraction:

$$v_2 = (0, -2, -2)$$

The third vector for the orthogonal subspace is a projection onto v_1 and v_2.

$$v_3 = x_3 - \frac{x_3 v_1}{v_1 v_1} v_1 - \frac{x_3 v_2}{v_2 v_2} v_2$$

Substitute the values:

$$v_3 = (5, 2, 0) - \frac{(5, 2, 0)(1, 2, -2)}{(1, 2, -2)(1, 2, -2)}(1, 2, -2) - \frac{(5, 2, 0)(1, 0, -4)}{(1, 0, -4)(1, 0, -4)}(1, 0, -4)$$

$$v_3 = (5, 2, 0) - (9/9)(1, 2, -2) - (-4/8)(1, 0, -4)$$

$$v_3 = (5, 2, 0) - (1, 2, -2) + (1/2)(1, 0, -4)$$

$$v_3 = (5, 2, 0) - (1, 2, -2) + (0, -1, -1)$$

$$v_3 = (4, -1, 1)$$

This set of vectors is orthogonal, so we need to normalize them so that the vectors are orthonormal. Recall that an orthonormal vector has a length of 1 and is computed using this formula:

$$normalizedVector = vector / np.sqrt(np.sum(vector * *2))$$

Thus the normalized set of vectors is:

$$v_1 = (0.33, 0.67, -0.67)$$

$$v_2 = (0.0, -0.71, -0.71)$$

$$v_3 = (0.94, -0.24, 0.24)$$

Exercise 1 Gram-Schmidt Process

Use the Gram-Schmidt Process to find an orthogonal basis for the span defined by x_1, x_2 where:

$$x_1 = (1, 1, 1)$$

$$x_2 = (0, 1, 1)$$

Working Space

Answer on Page 67

1.3 The Gram-Schmidt Process in Python

Create a file called `vectors_gram-schmidt.py` and enter this code:

```
# import numpy to perform operations on vector
import numpy as np

# Find an orthogonal basis for the span of these three vectors
x1 = np.array([1, 2, -2])
x2 = np.array([1, 0, -4])
x3 = np.array([5, 2, 0])

# v1 = x1
v1 = x1
print("v1 = ",v1)

# v2 = x2 - (the projection of x2 on v1)
v2 = x2 - (np.dot(x2,v1)/np.dot(v1,v1))*v1
print("v2 = ", v2)

# v3 = x3 - (the projection of x3 on v1) - (the projection of x3 on v3)
v3 = x3 - (np.dot(x3,v1)/np.dot(v1,v1))*v1 - (np.dot(x3,v2)/np.dot(v2,v2))*v2
print("v3 =", v3)

# Next, normalize each vector to get a set of vectors that is both orthogonal and
↪  orthonormal:
v1_norm = v1 / np.sqrt(np.sum(v1**2))
v2_norm = v2 / np.sqrt(np.sum(v2**2))
v3_norm = v3 / np.sqrt(np.sum(v3**2))
print("v1_norm = ", v1_norm)
print("v2_norm = ", v2_norm)
```

```
27 print("v3_norm = ", v3_norm)
```

1.4 Where to Learn More

Watch this video from Khan Academy about the Gram-Schmidt Process: https://www.youtube.com/watch?v=rHonltF77zI

Chapter 2

Singular Value Decomposition

In the previous chapter, you learned how to calculate eigenvalues and eigenvectors. However, not every matrix has them. For those matrices, singular values and singular vectors are analogous features.

Singular Value Decomposition (SVD) is a matrix factorization technique that breaks down a matrix into three matrices that represent the structure and properties of the original matrix. The decomposed matrices make calculations easier and provide insight into the original matrix. Essentially, SVD can transform a high dimension, highly variable set of data into a set of uncorrelated data points that reveal subgroupings that you might not have noticed in the original data. SVD tells us that a linear transformation can be thought of as a rotation, scaling, and another rotation.

2.1 Definition

For any $m \times n$ matrix A, SVD decomposes the matrix into three matrices.

$$A = U\Sigma V^T \tag{2.1}$$

- U is an orthogonal matrix whose size is $m \times m$. Its columns are the eigenvectors of AA^T. These are the left singular vectors of A. Because U is orthogonal, $U^TU = I$.
- V is an orthogonal matrix whose size is $n \times n$ matrix. Its columns are the eigenvectors of A^TA. These are the right singular vectors of A. Because V is orthogonal, $V^TV = I$.
- Σ is a diagonal matrix that is the same size as A. Its diagonal contains the singular values of A, arranged in descending order. These values are the square roots of the eigenvalues of both A^TA and AA^T.

2.2 Applications of SVD

SVD has numerous applications:

- It is used in machine learning and data science to perform dimensionality reduction, particularly through a technique known as Principal Component Analysis (PCA).

- In numerical linear algebra, SVD is used to solve linear equations and compute matrix inverses in a more numerically stable way.
- It is used in image compression, where low-rank approximations of an image matrix provide a compressed version of the original image.

2.3 Calculating SVD Manually

You might be inclined to skip this example because the computations are lengthy. Why would anyone do this when they can use a computing language, like Python, to calculate the SVD with essentially one command? We show this so you can understand what goes on "under the hood" when you compute SVD programmatically.

After you read through this example, you will see how to use Python to compute SVD. Next, you will see an example of using SVD for image compression. Finally, you will be given an exercise to compute the SVD. For this, you will need to write your own Python script.

Let's find the SVD for matrix A. Recall that we want to find U. Σ, and V^T such that:

$$A = U\Sigma V^T \tag{2.2}$$

$$A = \begin{bmatrix} 3 & 1 & 1 \\ -1 & 3 & 1 \end{bmatrix}$$

$$U = AA^T$$

Calculating

$$AA^T$$

will give us a square matrix:

$$A^T = \begin{bmatrix} 3 & -1 \\ 1 & 3 \\ 1 & 1 \end{bmatrix}$$

$$AA^T = \begin{bmatrix} 3 & 1 & 1 \\ -1 & 3 & 1 \end{bmatrix} \begin{bmatrix} 3 & -1 \\ 1 & 3 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 11 & 1 \\ 1 & 11 \end{bmatrix}$$

Next, we will find the eigenvalues and eigenvectors of A^T. This is a chance to apply what you learned in the previous chapter. We know that:

$$Av = \lambda v \tag{2.3}$$

So:

$$\begin{bmatrix} 11 & 1 \\ 1 & 11 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \lambda \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

Rewrite as a set of equations:

$$11x_1 + x_2 = \lambda x_1$$

$$x_1 + 11x_2 = \lambda x_2$$

Then rearrange:

$$(11-\lambda)x_1 + x_2 = 0$$

$$x_1 + (11-\lambda)x_2 = 0$$

Solve for λ:

$$\begin{bmatrix} (11-\lambda), 1 \\ 1, (11-\lambda) \end{bmatrix} = 0$$

And as equations:

$$(11-\lambda)(11-\lambda) - 1 \cdot 1 = 0$$

$$(\lambda - 10)(\lambda - 12) = 0$$

These are the eigenvalues.

$$\lambda = 10$$

$$\lambda = 12$$

When substituted into the original equations, you get the eigenvectors. For

$$\lambda = 10$$

:

$$(11-10)x_1 + x_2 = 0$$

$$x_1 = -x_2$$

We will set

$$x_1$$

to 1 and get this eigenvector:

$$[1, -1]$$

For

$$\lambda = 10$$

:

$$(11-12)x_1 + x_2 = 0$$

$$x_1 = x_2$$

We will set

$$x_1$$

to 1 and get this eigenvector:

$$[1, 1]$$

The matrix is:

$$\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

Next, you need to apply the Gram-Schmidt process to the column vectors. After that, you will have U, the $m \times m$ matrix whose columns are eigenvectors of AA^T. These are the left singular vectors of A. After you apply Gram-Schmidt, you should end up with:

$$U = \begin{bmatrix} 1/\sqrt{2} & 1/\sqrt{2} \\ 1/\sqrt{2} & -1/\sqrt{2} \end{bmatrix}$$

The process for calculating V is the same as the calculation for U, except:

$$V = A^T A$$

$$A^T A = \begin{bmatrix} 3 & -1 \\ 1 & 3 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 3 & 1 & 1 \\ -1 & 3 & 1 \end{bmatrix} = \begin{bmatrix} 10 & 0 & 2 \\ 0 & 10 & 4 \\ 2 & 4 & 2 \end{bmatrix}$$

After applying the process we applied to solve for U, you get:

$$V = \begin{bmatrix} 1/\sqrt{6} & 2/\sqrt{5} & 1/\sqrt{30} \\ 2/\sqrt{6} & -1/\sqrt{5} & 2/\sqrt{30} \\ 1/\sqrt{6} & 0 & -5/\sqrt{30} \end{bmatrix}$$

However, you want V_T:

$$V_T = \begin{bmatrix} 1/\sqrt{6} & 2/\sqrt{6} & 1/\sqrt{6} \\ 2/\sqrt{5} & -1/\sqrt{5} & 0 \\ 1/\sqrt{30} & 2/\sqrt{30} & -5/\sqrt{30} \end{bmatrix}$$

You have only to calculate Σ, a diagonal matrix that is the same size as A. The diagonal contains the singular values of A, arranged in descending order. These are the square roots of the eigenvalues of both $A^T A$ and AA^T.

Because the non-zero eigenvalues of U are the same as V, let's use the eigenvalues we calculate for U, 10 and 12. Note that Σ will not be of the correct dimension to reconstruct the original matrix unless we add a column. By adding a zero column you'll be able to multiply between U and V:

$$\Sigma = \begin{bmatrix} \sqrt{12} & 0 & 0 \\ 0 & \sqrt{12} & 0 \end{bmatrix}$$

You can check your work by multiplying the decomposed matrices. This should return the original matrix.

$$A = U\Sigma V^T$$

$$= U = \begin{bmatrix} 1/\sqrt{2} & 1/\sqrt{2} \\ 1/\sqrt{2} & -1/\sqrt{2} \end{bmatrix} \begin{bmatrix} \sqrt{12} & 0 & 0 \\ 0 & \sqrt{12} & 0 \end{bmatrix} \begin{bmatrix} 1/\sqrt{6} & 2/\sqrt{6} & 1/\sqrt{6} \\ 2/\sqrt{5} & -1/\sqrt{5} & 0 \\ 1/\sqrt{30} & 2/\sqrt{30} & -5/\sqrt{30} \end{bmatrix}$$

$$= \begin{bmatrix} \sqrt{12}/\sqrt{2} & \sqrt{10}/\sqrt{2} & 0 \\ \sqrt{12}/\sqrt{2} & -\sqrt{10}/\sqrt{2} & 0 \end{bmatrix} \begin{bmatrix} 1/\sqrt{6} & 2/\sqrt{6} & 1/\sqrt{6} \\ 2/\sqrt{5} & -1/\sqrt{5} & 0 \\ 1/\sqrt{30} & 2/\sqrt{30} & -5/\sqrt{30} \end{bmatrix}$$

$$= \begin{bmatrix} 3 & 1 & 1 \\ -1 & 3 & 1 \end{bmatrix}$$

2.4 Singular Value Decomposition with Python

Create a file called `vectors_decomposition.py` and enter this code:

```
# Singular-value decomposition
import numpy as np
from numpy import array
from scipy.linalg import svd
from numpy import diag
from numpy import dot
from numpy import zeros

# Define a matrix
A = array([[1, 2], [3, 4], [5, 6]])

print("Matrix (3x2) to be decomposed: ")
print(A)

# CalculateSVD
U, S, VT = svd(A)
print("Matrix (3x3) that represents the left singular values of A:")
print(U)
print("Singular values:")
print(S)
print("Matrix (2x2) that represents the right singular values of A:")
print(VT)

# Check if the decomposition by rebuilding the original matrix
# The singular values must be in an m x n matrix
# Create a zero matrix with the same dimension as A
Sigma = zeros((A.shape[0], A.shape[1]))
# Populate Sigma with n x n diagonal matrix
Sigma[:A.shape[1], :A.shape[1]] = diag(S)
# Reconstruct the original matrix
A_Rebuilt = U.dot(Sigma.dot(VT))
print("Original matrix:")
print(A_Rebuilt)
```

2.5 Sign Ambiguity

You might notice that at times, the absolute values in the U and V^T matrices are correct, but that the signs vary from what you see as the answer. For example, when you compare a manually calculated SVD with one done in Python the signs might not agree. Both decompositions of A are valid. Both decompositions will satisfy:

$$A = U\Sigma V^T$$

Note that the S diagonal values will always be positive.

The sign ambiguity has implications. For example, when using SVD to compress data, if some of the signs are flipped, the data can have artifacts. At this point in your education, you don't need to concern yourself with it, except when you are comparing SVD results for the same matrix.

Exercise 2 Single Value Decomposition

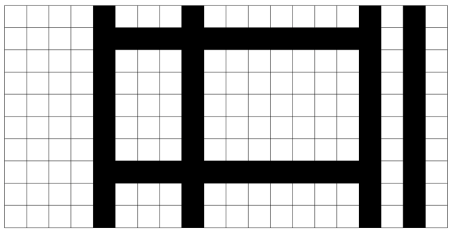
Modify your Python code to calculate SVD for the matrix in the worked out example. Did you arrive at the same answer? Keep in mind that Python will compute square roots and present fractions as decimal. Take a look at the signs for the values in the U and V^T matrices. Are they the same, or is this an example of sign ambiguity?

Working Space

Answer on Page 67

2.6 SVD Applied to Image Compression

This image consists of a grid of 20 by 10 pixels, each of which is either black or white.



It is a simple image that has only two types of columns–ideal for data compression. A row is either the first pattern or the second.



We can represent the data as a 20 by 10 matrix whose 200 entries are either 0 for black or 1 for white.

$$\begin{bmatrix}
00001000100000001010\\
00001111111111111010\\
00001000100000001010\\
00001000100000001010\\
00001000100000001010\\
00001000100000001010\\
00001000100000001010\\
00001111111111111010\\
00001000100000001010\\
00001000100000001010
\end{bmatrix}$$

When you perform an SVD on this matrix, there are only two non-zero singular values, 6.79 and 3.72. (You are welcome perform the calculation in Python.) Thus, you can represent the matrix as:

$$A = U_1 S_1 V_1 + U_2 S_2 V_2$$

This means there are two u vectors, each with 20 entries, two v vectors each with 10 entries, and two singular values. Add those up: 2*20 + 2*10 + 2 = 62. This implies that the image can be represented by 62 values instead of 200. If you look back at the image, you can see that there are many dependent columns and very few independent ones.

This is a simple image and a small pixel matrix, but it should give you a sense of how SVD can decompose an image in a way that identifies how much of the image is redundant, and therefore can be compressed.

2.7 Where to Learn More

We Recommend a Singular Value Decomposition. This American Mathematical Society publication focuses the geometry of SVD. What we like about the article is that it shows both

graphically and numerically how SVD can be used for data compression on images and for noise reduction. The data compression example in your workbook is based on this article. https://www.ams.org/publicoutreach/feature-column/fcarc-svd

Sign Ambiguity in Singular Value Decomposition (*SVD*). This is a good article for those who want a deeper understanding of sign ambiguity. https://www.educative.io/blog/sign-ambiguity-in-singular-value-decomposition

Singular Value Decomposition Tutorial. This PDF starts by defining points, space, and vectors and works through all the concepts you need to tackle SVD. It is one of the few resources that has a completely worked out example of manually calculating SVD. The example in this chapter is from that tutorial. If you read the entire paper, you will find that it is a good review of the concepts you have studied in previous chapters. https://rb.gy/j6s0w

Chapter 3

Tackling Difficult Problems: Positive Semidefinite Matrices

With all the computing power available today, you would think no problem would be too difficult to tackle. However, this is not the case. There is a category of problems called NP (nondeterministic polynomial time) whose solution is easy to verify, but whose computation is difficult to perform. This is because there is no straightforward algorithm and any brute-force method would take too much time. For these problems, all we can do is to develop an efficient algorithm that can find a solution in a reasonable amount of time. While the solution might not be the most optimal, the goal is to find the best solution possible in a short time.

As you learn more about optimization techniques, you will come across many efficiency algorithms that have been used throughout the years. In the 1990s, the field of optimization changed with the discovery that algorithms based on semidefinite positive matrices can achieve a higher efficiency than seen in the past. Today, there is an entire field of programming called Semidefinite Programming that is based on the use of semidefinite positive matrices.

First we will take a look at what some of the NP problems are. Next, we will describe the intuition behind semidefinite positive matrices. We will take a look at one NP problem, then introduce the Python module to use for solving these problems.

3.1 NP Problems

In the world of mathematics, easy problems are referred to as P, or polynomial time, problems. In simple terms, this means the problem can be solved quickly, and it is easy to verify that the solution is correct. Addition, subtraction, division, multiplication, square roots, and matrix-vector multiplication are just a few examples. NP problems, as stated in the introduction, can't be computed in a reasonable amount of time, but solutions are usually easy to verify. The game of Sudoku is one such example. It is easy to verify a correct solution, but writing a generalized algorithm to solve any Sudoku game is an NP problem.

NP problems show up in many other situations, such as:

- Designing robust communication networks

- Scheduling tasks without conflicts
- Managing supply chains
- Detecting patterns in biological networks
- Figuring out subgroups in social networks
- Predicting the structure of proteins

For each of these , think of large scale problems for which there are many variables. A cloud computing company that provides AI services to thousands of clients must be able to schedule tasks efficiently and in a timely manner, as well as manage the power needed for the computers and cooling the data center. Supply chain management is crucial to figuring out how to pick up, transport, and distribute goods to help provide disaster relief. Understanding protein structure is important for designing drugs that can tackle specific conditions. All we can do for each of these situations is to find an optimal solution, but we cannot find the definitive solution.

3.2 A Past Approach: Minimizing Errors in Neural Networks

When neural networks were first being developed to recognize things (faces, letters, music, and so on), the goal was to calculate weights between network nodes that would minimize the recognition error. The error space could be visualized as a surface of valleys and hills. The lowest point would have the least error. The idea behind the iterative weight calculations for training the network was to descend down the gradient until reaching the low point. Without getting into the mathematics, you can see by looking at this figure that there are two valleys. Some neural network training resulted in ending at a low point, but not the lowest point. Wouldn't it be great if you could formulate an optimization problem so that you would be guaranteed to land at the lowest point? That is where semidefinite programming can help.

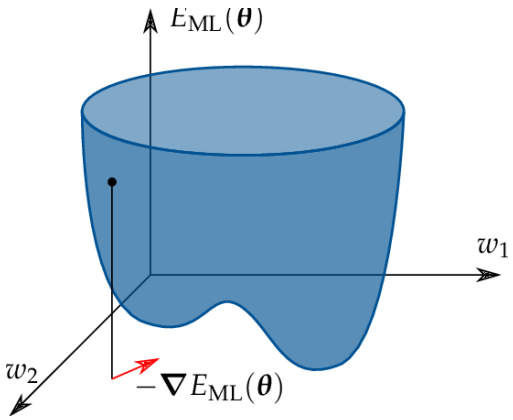
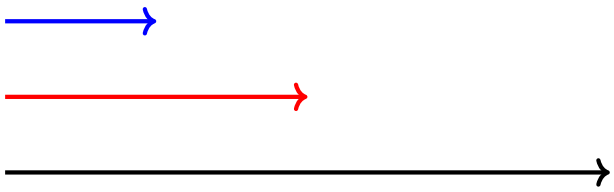


Figure 3.1: A neural network error surface with two valleys. Gradient descent can converge to a local minimum rather than the global minimum.

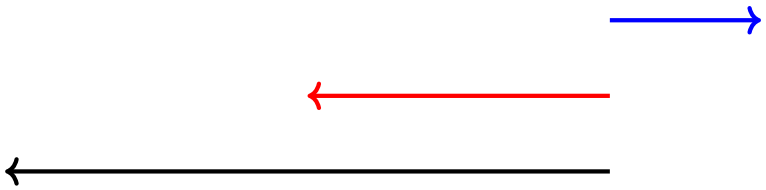
3.3 Positive Definite and Semidefinite Matrices

Unlike matrices that are defined by their content (such as identity matrix, zero matrix, and diagonal matrix), positive definite and semidefinite matrices are characterized by the result they produce. They have an analog in the scalar world, so let's first look at that. Let's take the scalars a and b and treat them as vectors. ab is then the dot product. If $a > 0$, then ab will take on the sign of b. If $a < 0$, then ab will have the opposite sign of b. If you look at this in a graph, you can see that in the first case, ab stays on the same side of the origin, but in the second case, ab flips

For $a = [2], b = [4]$



For $a = [3], b = [-4]$. the result of multiplication flips a to the other side of the graph.



The notion of "staying on the same side" is positive definite. The notion of flipping is negative definite. Positive definite means that $x > 0$, so the result is a positive number. Positive semidefinite means that $x >= 0$, so the result is a non-negative number.

If you can formulate a problem as a positive semidefinite matrix, then you automatically constrain the result to the "same side." This constraint results in higher algorithmic efficiency.

Let's look at the formal definition:

A matrix is positive definite if, and only if:

$$x^T Ax > 0, x, x \neq 0$$

A matrix is positive semidefinite, if, and only if:

$$x^T Ax \geq 0$$

Further, a positive semidefinite matrix had an important property. Its eigenvalues are ≥ 0. The eigenvalues of a positive definite matrix are > 0.

Take the triplet (a, b, c) and the symmetric matrix:

$$\begin{bmatrix} 1 & a & b \\ a & 1 & c \\ b & c & 1 \end{bmatrix} \geq 0$$

For what values of a, b, c is this matrix positive semidefinite? If you iterate through all possible combinations of a,b,c, then compute the eigenvalues for each matrix, you will find that some (like $0, 0, 0$) result in a positive semidefinite matrix and some (like $2, 2, 2$) are not positive semidefinite. If you plot the set of triplets that result in a semidefinite matrix, you will see an elliptope. This shape guarantees an optimal solution.

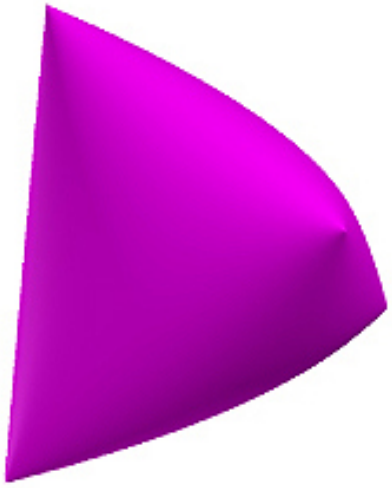


Figure 3.2: Elliptope visualized.

3.4 Identifying and Constructing a Positive Semidefinite Matrix

In the last section, you saw that being symmetric does not guarantee a positive semidefinite matrix. You also saw that a matrix of positive values does not guarantee a positive semidefinite matrix. The only way to check for positive semidefinite is to calculate the eigenvalues, which you learned in a previous workbook.

A surefire way to construct a positive semidefinite matrix is:

$$AA^T$$

Exercise 3 Figuring out if a matrix is positive semidefinite

Is this matrix positive definite? Show your work.

$$\begin{bmatrix} 2 & 2 \\ 2 & 0 \end{bmatrix}$$

Working Space

Answer on Page 68

Exercise 4 Creating a positive semidefinite matrix

Using any 3 by 3 matrix, create a positive semidefinite matrix.Then show it is positive semidefinite by calculating its eigenvalues. You can either compute this by hand or using Python. In either case, show your work.

Working Space

Answer on Page 68

3.5 The Max Cut Problem

A famous NP problem is Max Cut. Given a graph of interconnected nodes, cut the graph to create two sets of nodes, such that the cut goes through as many edges as possible. (You can't cut an edge more than once.) Max Cut is important for binary classification in machine learning, circuit design, statistical physics, and more. There is no algorithm that will provide an exact solution. (If you could find one, you would be eligible to win a huge prize from the Clay Mathematics Institute!) Instead, you will see how to approximate a solution to this problem using a positive semidefinite matrix and a technique developed by mathematicians Michel Goemans and David Williamson.

You won't see all the complete details here, as this section is meant to be a quick introduction to how you can apply positive semidefinite matrices.

Take this simple graph of five nodes and six edges. Each node in the graph will take on

one of two values (1 or -1) to show which set they fall into after a cut is made. For any two connected nodes, $x_i \cdot x_j = 1$ if $x_i = x_j$ and -1 otherwise.

If you randomly assign 1 and -1 to the nodes, the chance of making the max cut is 0.5. By using semidefinite programming, you can achieve an algorithmic efficiency of 0.87.

The Goemans-Williamson technique can be used for any optimization problem where the variables take on the values of 1 and -1.

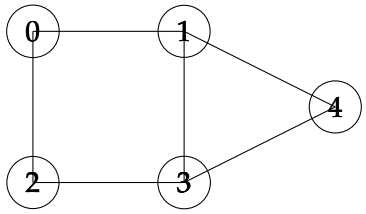


Figure 3.3: MaxCut algorithm visualized.

As a list of edges the graph is:

$$edges = [(0,1),(0,2),(1,3),(1,4),(2,3),(3,4)]$$

The optimization problem can be formulated as:

$$Max \sum_{edges(i,j)} \frac{1 - x_i x_j}{2}$$

for

$$x_i \in \{-1, 1\}$$

However, instead of allowing x_i to be scalar, Goemans-Williamson defines x_i as unit vectors.

$$x_i \in R^n, x_i = 1$$

and that makes the optimization equation:

$$Max \sum_{edges(i,j)} \frac{1 - x_i^T x_j}{2}$$

It is this "relaxation" that gets us to a semidefinite matrix, because we can now rewrite

the problem as a positive semidefinite matrix:

$$X = \left[x_i^T x_j\right]_{i,j}$$

Python has a module for solving optimization problems. Using this, you will get an optimum matrix, but to get the unit vectors, you will need to take the square root of the matrix.

$$X = [x_1 ... x_{1n}]\,[x_1 ... x_{1n}]^T$$

Next we need to go from unit vectors to scalars, using a process called rounding.

$$x^i \in R^n \rightarrow x^i \in \{-1, 1\}$$

Goemans-Williamson leveraged the fact that the end point of a unit vector is on a sphere. They generated a random plane to bisect the sphere. A vector on one side of the plane is assigned the value of 1 and a vector on the other side a value of -1.

You can then assign the scalar values to the nodes and make the cut accordingly. This particular cut will be 5, as shown.

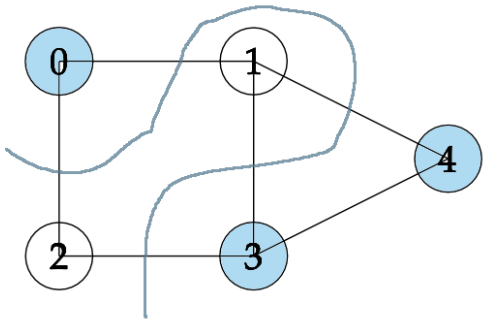


Figure 3.4: The result of the MaxCut.

3.6 The Max Cut Problem Solved in Python

```
# MaxCut Problem
import numpy as np
import scipy
from scipy.linalg import sqrtm
# cvxpy is a python module for solving optimization problems
import cvxpy as cp

# define the edges of the graph
edges = [(0,1),
         (0,2),
         (1,3),
         (1,4),
```

```
        (2,3),
        (3,4)]

#  Declare the matrix X to be positive semidefinite
X = cp.Variable((5,5),symmetric=True)
constraints = [X >> 0]

# Set diagonals to 1 to get unit vectors
constraints += [
    X[i,i] == 1 for i in range(5)
]

# Set the objective function
objective = sum(05.*(1 - X[i,j]) for (i,j)in edges)

# Set up the problem to maximize using the objective function and
# keeping within the set constraints
prob = cp.Problem(cp.Maximize(objective), constraints)

# Returns a positive semidefinite matrix
print(prob.solve())

# To get the vectors, take square root of the matrix
x = sqrtm(X.value)

# Generate a random hyperplane
u = np.random.randn(5) # normal to random hyperplane

# Pick values according to which side of the hyperplane the vectors are on
x = np.sign(x @ u)

```

CHAPTER 4

Introduction to C

You already know how to write programs in Python. In the next few chapters, you are going to learn C. Your Python knowledge will make this easier, because C has the same fundamental ideas: variables, functions, loops, and conditionals. The syntax looks different, and there are a few genuinely new concepts, but you will recognize the shape of everything you write.

Why learn C? Two reasons. First, C is used wherever performance matters: operating systems, game engines, embedded systems, and scientific computing. Second, and more important for this course, C forces you to think about memory explicitly. Python quietly manages memory on your behalf; C makes you do it yourself. Once you understand how C uses memory, you will also understand what Python is doing underneath, and you will be ready to implement data structures from scratch.

4.1 Compiled vs. Interpreted

When you run a Python program, the Python interpreter reads your `.py` file and executes it line by line. There is no separate step between writing and running.

C works differently. Before a C program can run, a *compiler* must translate it into machine code, the raw instructions that your CPU executes directly. The compiler reads your `.c` file, checks it for errors, and produces an *executable* file. You then run that executable.

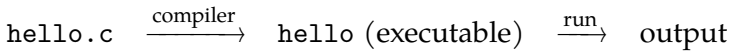


Figure 4.1: The compile-then-run cycle

Because the compiler sees your entire program before anything runs, it can catch many mistakes, like using a variable before declaring it or passing the wrong type to a function, that Python would only discover at runtime.

To compile a C file called `hello.c`, you run:

```
1 gcc hello.c -o hello
```

This `gcc` is the compiler that turns your C code into an executable. The `hello.c` is the input file, and `-o hello` tells the compiler to name the output executable `hello`. On

macOS and Linux systems, this may come pre installed. On Windows, however, this likely is not installed. You can find the installation instructions here.

The `-o hello` flag names the output executable `hello`. Then you run it:

```
./hello
```

4.2 Your First C Program

Create a file called `hello.c` and type the following:

```c
#include <stdio.h>

int main()
{
    printf("Hello, world!\n");
    return 0;
}
```

Compile and run it. You should see:

```
Hello, world!
```

Let's look at each piece.

`#include <stdio.h>` tells the compiler to include the standard input/output library, which provides `printf` for printing. Think of it like Python's `import` statement.

`int main()` is the function where your program starts. Every C program must have exactly one `main` function. The `int` means it returns an integer; `return 0` signals to the operating system that the program finished successfully.

`printf("Hello, world!\n");` prints a line of text. The `\n` is the newline character.

Notice the semicolons at the end of statements, and the curly braces `{ }` around the body of `main`. In Python, indentation defines blocks; in C, curly braces do.

4.3 Types and Variables

In Python, you write:

```
x = 5
y = 3.14
```

Python figures out the type of each variable automatically.

In C, you must declare the type of every variable when you create it:

```
int x = 5;
double y = 3.14;
```

This is called *static typing*. Once you declare a variable as `int`, it can only hold integers. The compiler enforces this and will refuse to compile code that mixes types incorrectly.

The common C types are:

`int` A whole number: `0`, `42`, `-7`. Typically 4 bytes.

`double` A floating-point number: `3.14`, `-0.001`. Typically 8 bytes. (There is also `float`, which uses 4 bytes and has less precision.)

`char` A single character: `'A'`, `'z'`, `'7'`. Use single quotes.

`int` Booleans in C are just integers: `0` means false, and any non-zero value means true. You can include `<stdbool.h>` to get the names `true` and `false`.

You can also declare a variable without immediately giving it a value:

```
int count;      /* declared but not initialized */
count = 10;     /* now it has a value */
```

Warning: an uninitialized variable in C contains whatever random bytes happen to be in that memory location. Unlike Python, C will not raise an error if you read an uninitialized variable; it will just give you garbage. Always initialize your variables.

Note that C uses `/* ... */` for comments, not `#`. You can also use `//` for single-line comments in modern C.

4.4 Printing with `printf`

`printf` uses format specifiers to print different types:

```c
int age = 17;
double height = 1.75;
printf("Age: %d, Height: %f\n", age, height);
```

Output:

```
Age: 17, Height: 1.750000
```

Common format specifiers:

Specifier	Type
%d	int
%f	double or float
%c	char
%s	string (char pointer)

4.5 Conditionals and Loops

The logic is the same as Python; only the syntax changes.

4.5.1 if / else

```c
int score = 85;

if (score >= 90) {
    printf("A\n");
} else if (score >= 80) {
    printf("B\n");
} else {
    printf("C or below\n");
}
```

The condition must be in parentheses `( )`, and each branch is surrounded by curly braces `{ }`.

4.5.2 for and while loops

```c
for (int i = 0; i < 5; i++) {
    printf("%d\n", i);
}
```

A C `for` loop has three parts separated by semicolons: the initializer (`int i = 0`), the condition (`i < 5`), and the update (`i++`). The `++` increments by 1, the same as `i = i + 1`.

```c
int i = 0;
while (i < 5) {
    printf("%d\n", i);
    i++;
}
```

4.6 Functions

In C, you must declare the return type and the type of each parameter:

```c
double square(double x)
{
    return x * x;
}

int main()
{
    printf("%f\n", square(4.0));   /* prints 16.000000 */
    printf("%f\n", square(2.5));   /* prints 6.250000  */
    return 0;
}
```

If a function returns nothing, its return type is `void`:

```c
void printGreeting(char *name)
{
    printf("Hello, %s!\n", name);
}
```

Functions must be defined *before* they are called, or you must provide a *forward declaration* (just the signature, ending in a semicolon) above `main`:

```c
double square(double x);   /* forward declaration */

int main()
{
    printf("%f\n", square(3.0));
    return 0;
}

double square(double x)    /* full definition can come later */
{
    return x * x;
}
```

4.7 The Stack: Blackboards for Functions

Every function, while it is running, needs somewhere to store its local variables. C gives each function a *frame*: a reserved chunk of memory that holds all of that function's local variables and parameters.

Think of a frame as a **blackboard**. When a function starts running, it gets a fresh blackboard. It can write anything it wants on that blackboard, create variables, change their values. When the function finishes, the blackboard is *erased and discarded*. Local variables do not survive after their function returns.

Where do these frames live? In a region of memory called the *stack*. The stack works exactly like a physical stack of objects: the most recently added item is always on top. When `main` calls a function, that function's frame is pushed onto the top of the stack. When the function returns, its frame is popped off, and `main`'s frame is on top again.

Here is a concrete example:

```c
#include <stdio.h>

int cookingMinutes(int weightPounds)
{
    int minutes = 15 + 15 * weightPounds;
    return minutes;
}

int main()
{
    int totalWeight = 10;
    int gibletsWeight = 1;
    int turkeyWeight = totalWeight - gibletsWeight;
    int result = cookingMinutes(turkeyWeight);
    printf("Cook for %d minutes.\n", result);
```

```
16      return 0;
17  }
```

When `main` calls `cookingMinutes(9)`, the stack looks like this:

cookingMinutes()	`weightPounds = 9` `minutes = 150`

main()	`totalWeight = 10`
	`gibletsWeight = 1`
	`turkeyWeight = 9`
	`result = ?`

`main`'s blackboard sits below; `cookingMinutes`'s blackboard sits on top of it. The variable `minutes` inside `cookingMinutes` is completely separate from anything in `main`. Each function only sees its own blackboard.

When `cookingMinutes` returns 150, its frame is discarded. `main`'s frame resumes, and `result` is filled in with 150.

4.7.1 Parameters are copies

When you call `cookingMinutes(turkeyWeight)`, C copies the value of `turkeyWeight` into the new parameter `weightPounds` on `cookingMinutes`'s blackboard. Changing `weightPounds` inside the function would not change `turkeyWeight` in `main`. They are separate variables. This is called *pass-by-value*.

4.8 Addresses and Pointers

Every variable in your program lives at some address in memory. You can find out the address of a variable using the `&` operator, called the *address-of* operator:

```
1   #include <stdio.h>
2
3   int main()
4   {
5       int i = 17;
6       printf("i stores its value at %p\n", (void *)&i);
7       return 0;
8   }
```

Run this and you will see something like:

```
i stores its value at 0x7ffeea3c4738
```

The address is printed in hexadecimal. Your address will differ, because the operating system decides where in memory your variables live.

A *pointer* is a variable that holds an address. If you want to store the address of an `int`, you declare a variable of type `int *` ("pointer to int"):

```
int i = 17;
int *addressOfI = &i;
printf("i is at %p\n", (void *)addressOfI);
```

The `*` in the declaration is part of the type; it means "this variable holds an address, and the thing at that address is an `int`."

4.8.1 Reading and writing through a pointer

Once you have a pointer, you can use the `*` operator to read or write the value *at* the address the pointer holds. This is called *dereferencing* the pointer:

```
int i = 17;
int *p = &i;

printf("%d\n", *p);   /* prints 17 -- reads through p  */

*p = 89;              /* writes 89 to the address p holds */
printf("%d\n", i);    /* prints 89 -- i was changed!   */
```

To summarize:

- `&i` gives you the address of `i`.
- `int *p` declares a variable that can hold such an address.
- `*p` gives you the value stored at the address `p` holds.

4.8.2 NULL pointers

Sometimes you want a pointer that does not yet point to anything. Use `NULL`:

```c
int *p = NULL;   /* p points to nothing */

if (p) {
    printf("%d\n", *p);     /* only runs if p is non-null */
} else {
    printf("p is null\n");
}
```

A null pointer is simply the address zero. **Never dereference a null pointer**, your program will crash. Always check before dereferencing a pointer that might be null.

4.9 Pass-by-Reference

Recall that when you call a function in C, arguments are *copied* into the function's frame. But what if you want a function to modify a variable in the calling function? You pass the *address* of that variable instead of its value. The function can then write to that address.

There is a standard C function called `modf()` that splits a floating-point number into its integer part and its fractional part, and needs to return *both*. Since a function can only return one value, it returns the fractional part and writes the integer part to an address you supply:

```c
#include <stdio.h>
#include <math.h>

int main()
{
    double pi = 3.14;
    double integerPart;
    double fractionPart;

    /* Pass the address of integerPart so modf can write there */
    fractionPart = modf(pi, &integerPart);

    printf("Integer part:  %f\n", integerPart);
    printf("Fraction part: %f\n", fractionPart);
    return 0;
}
```

Output:

```
Integer part:  3.000000
Fraction part: 0.140000
```

Here is another way to think about it. Imagine you need to give a spy an assignment. You say: "I've left a length of steel pipe at the foot of the angel statue in the park. When you have the information, roll it up and leave it in the pipe. I'll pick it up Tuesday." In the spy business, this is called a *dead drop*.

Pass-by-reference works the same way. You are not giving the function the data directly, you are giving it a *location* where it can leave the result.

Here is our own pass-by-reference function that converts meters to feet and inches:

```c
#include <stdio.h>
#include <math.h>

void metersToFeetAndInches(double meters,
                           int *ftPtr,
                           double *inPtr)
{
    double rawFeet = meters * 3.281;
    int feet = (int)floor(rawFeet);
    double inches = (rawFeet - feet) * 12.0;

    if (ftPtr) *ftPtr = feet;
    if (inPtr) *inPtr = inches;
}

int main()
{
    int feet;
    double inches;
    metersToFeetAndInches(1.8, &feet, &inches);
    printf("%d ft %f in\n", feet, inches);
    return 0;
}
```

The caller declares the variables it wants filled in (`feet`, `inches`), passes their addresses to the function, and the function writes the results there.

4.10 Structs: Grouping Related Data

A *struct* lets you bundle several related values into a single named type. In Python you might use a dictionary for this; in C, a struct is the fundamental grouping mechanism.

```
struct Person {
    double heightInMeters;
    int weightInKilos;
};
```

This defines a new type called `struct Person`. You can then create variables of that type:

```
#include <stdio.h>

struct Person {
    double heightInMeters;
    int weightInKilos;
};

double bodyMassIndex(struct Person p)
{
    return p.weightInKilos / (p.heightInMeters * p.heightInMeters);
}

int main()
{
    struct Person mikey;
    mikey.heightInMeters = 1.70;
    mikey.weightInKilos  = 96;

    struct Person aaron;
    aaron.heightInMeters = 1.97;
    aaron.weightInKilos  = 84;

    printf("mikey BMI: %f\n", bodyMassIndex(mikey));
    printf("aaron BMI: %f\n", bodyMassIndex(aaron));
    return 0;
}
```

You access the members of a struct using a dot: `mikey.heightInMeters`.

When `main` is running, the stack looks like this:

main()	mikey.heightInMeters = 1.70 mikey.weightInKilos = 96 aaron.heightInMeters = 1.97 aaron.weightInKilos = 84

The two `Person` structs live right inside `main`'s frame on the stack.

4.11 The Heap

The stack is fast and automatic, but it has two constraints: stack frames are destroyed when functions return, and the sizes of all stack-allocated variables must be known at compile time.

The *heap* is a separate region of memory that has neither constraint. Memory on the heap persists until you explicitly release it, and can be any size determined at runtime.

In C, you claim heap memory with `malloc` and release it with `free`.

```
#include <stdlib.h>

struct Person *mikey = malloc(sizeof(struct Person));
```

`malloc` asks the system for enough heap memory to hold a `Person` struct and returns the address of that memory. The variable `mikey` is a *pointer* to a `Person`, it lives on the stack, but the struct it points to lives on the heap.

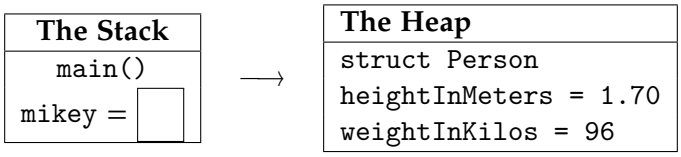


Figure 4.2: A pointer on the stack pointing to a struct on the heap

4.11.1 The arrow operator

When you have a pointer to a struct, you access its members using `->` instead of a dot:

```
mikey->heightInMeters = 1.70;
mikey->weightInKilos  = 96;
```

The code `mikey->weightInKilos` means: "dereference the pointer `mikey` to get the struct, then access its `weightInKilos` member." It is shorthand for `(*mikey).weightInKilos`.

4.11.2 Releasing heap memory

When you are done with heap-allocated memory, you must release it with `free`:

```
free(mikey);
mikey = NULL;   /* good habit: prevents accidental reuse */
```

After `free`, the memory is returned to the heap for reuse. Setting the pointer to `NULL` prevents you from accidentally using it again.

If you forget to call `free`, the memory is never returned. This is called a *memory leak*. In a short-lived program it may not matter, but in a long-running program it can consume all available memory.

4.11.3 A complete heap example

```
#include <stdio.h>
#include <stdlib.h>

struct Person {
    double heightInMeters;
    int weightInKilos;
};

double bodyMassIndex(struct Person *p)
{
    return p->weightInKilos / (p->heightInMeters * p->heightInMeters);
}

int main()
{
    struct Person *mikey = malloc(sizeof(struct Person));
    mikey->heightInMeters = 1.70;
    mikey->weightInKilos  = 96;

    printf("mikey BMI: %f\n", bodyMassIndex(mikey));

    free(mikey);
    mikey = NULL;

```

```
    return 0;
}
```

`bodyMassIndex` now takes a `struct Person *` instead of a `struct Person` by value. This avoids copying the struct. Functions that work on heap-allocated data almost always take pointers.

4.11.4 Heap arrays

You can also allocate a contiguous block of memory for multiple values. Use `malloc` with a count:

```
/* Allocate space for 1000 doubles on the heap */
double *buffer = malloc(1000 * sizeof(double));

buffer[0] = 3.14;
buffer[1] = 2.72;

free(buffer);
buffer = NULL;
```

A pointer to the first element is all you need to work with the whole array. The heap memory is contiguous, so `buffer[0]`, `buffer[1]`, and so on are laid out one after another.

4.12 What Python Objects Actually Are

You now know everything you need to understand what a Python object is, under the hood.

When Python creates an object, it:

1. Allocates a struct on the heap large enough to hold the object's data (its instance variables).
2. Stores in that struct a pointer to the class's method table, so that method calls can find the right code.
3. Returns a pointer to that struct.

Your Python variable `obj` is, at the machine level, a pointer to a heap-allocated struct. When Python's garbage collector determines that nothing points to an object anymore, it calls the equivalent of `free` to release that memory.

The next chapter introduces C++ classes, and with this memory model in hand, you will implement linked lists, trees, and hash tables from scratch.

4.13 Exercises

1. Write a C function `celsiusToFahrenheit` that takes a `double` temperature in Celsius and returns the equivalent in Fahrenheit. Call it from `main` for the values 0, 20, 37, and 100, and print each result.

2. Write a function `swap` that takes two `int` pointers and swaps the values at the two addresses. Test it so that after calling `swap(&a, &b)`, the values of `a` and `b` have been exchanged.

3. Write a struct `Point` with `double` members `x` and `y`. Write a function `distance` that takes two `struct Point *` pointers and returns the Euclidean distance between them. Allocate two points on the heap, fill in their coordinates, print the distance, and then free the memory.

4. **Memory leak hunt.** The following program has a memory leak. Identify and fix it.

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int value;
};

void printValue(int x)
{
    struct Node *n = malloc(sizeof(struct Node));
    n->value = x;
    printf("%d\n", n->value);
}

int main()
{
    for (int i = 0; i < 5; i++) {
        printValue(i);
    }
    return 0;
}
```

5. A `struct tm` is defined in `<time.h>` and holds a broken-down calendar time. The function `time(NULL)` returns the number of seconds since midnight, January 1, 1970. The function `localtime_r(&seconds, &now)` fills a `struct tm` with the corresponding date and time. Here is a snippet to get you started:

```c
#include <stdio.h>
#include <time.h>
```

```
3
4  int main()
5  {
6      long secondsSince1970 = time(NULL);
7      struct tm now;
8      localtime_r(&secondsSince1970, &now);
9      printf("The time is %d:%d:%d\n",
10             now.tm_hour, now.tm_min, now.tm_sec);
11     return 0;
12 }
```

Look up the other fields of `struct tm` and modify the program to print today's full date. Then modify it to print the date that falls exactly 4,000,000 seconds from now. (Hint: `tm_mon` is 0-indexed, so January is 0.)

Chapter 5

Linked Lists and C++ transition

5.1 From C to C++

In this chapter we are going to transition from stand C programming to C++. Why C++? Because it is a more modern version of C++ that allows for a deeper insight into the structures that we put into the memory of the computer. C++ differs in that it allows for object-oriented programming, which is a programming paradigm that uses ”objects” to represent data and methods. This allows for more complex data structures and algorithms to be implemented in a more organized and efficient manner. To do this, we will be utilizing *classes*, which we briefly covered in our Advanced Python chapter. In C++, classes are defined using the `struct` keyword.

There are two additional keyword differences you should be aware of regarding C vs C++. The primary one is the usage of the keyword *new*, which does not exist in C. In C, we have `malloc()`, which frees up a given amount of memory in bytes, and `sizeof`, which returns the size in bytes of a data structure.

```
1    // C : what you already know
2    struct Car *n = malloc(sizeof(struct Car));
```

C++ combines these two commands into one, called `new`.

```
1    // C++ : same thing, no sizeof, no cast
2    Car* n = new Car();
```

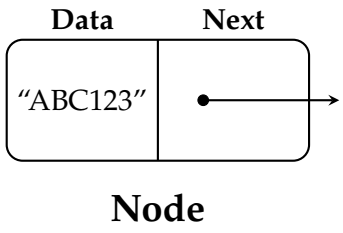
Here is a helpful video on some of the deeper differences that we will not be covering: https://www.youtube.com/watch?v=B_4x7WlQr7M

5.2 Linked Lists

A *linked list* is a linear data structure where each element is a separate object, called a *node*. Each node holds its own data and the address of the next node, called a *successor*, thus forming a chain-like structure. The first node in the list is called the *head*. The head is the only node you need to keep a direct reference to, since every other node can be reached by following successors from it. The last node, the one whose successor pointer is empty, is called the *tail*.

A simple node in a linked list can be represented in C++ as follows:

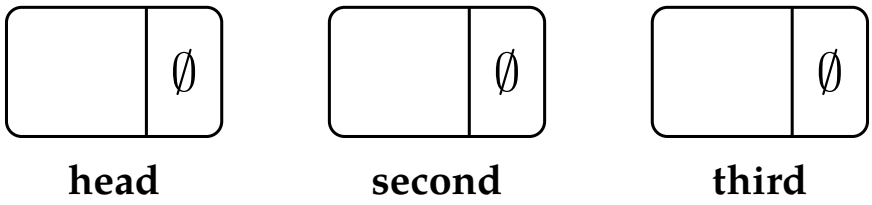
```
struct Node {
    int data;
    Node* next;
};
```



In this structure, 'data' is used to store the data and 'next' is a pointer that holds the address of the next Node in the list. When a pointer has nothing to point to, we mark its field with the empty-set symbol ∅ — the mathematical symbol for “nothing here.”

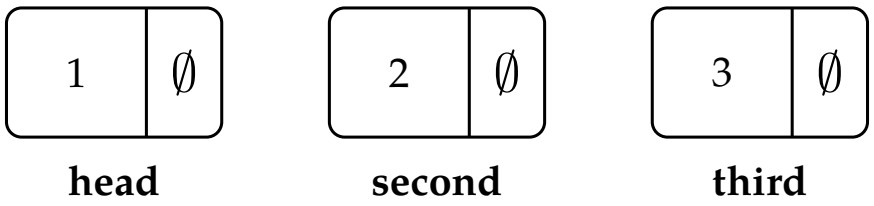
Here is a simple example of creating and linking nodes in a linked list:

```
// Create nodes
Node* head = new Node();
Node* second = new Node();
Node* third = new Node();
```

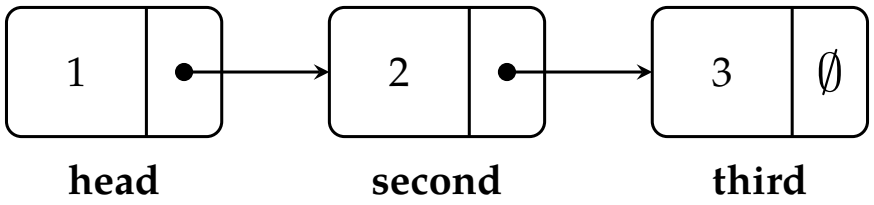



```

// Assign data
head->data = 1;
second->data = 2;
third->data = 3;
```




```
// Link nodes
head->next = second;
second->next = third;
third->next = nullptr;  // The last node points to null, this is the TAIL
```



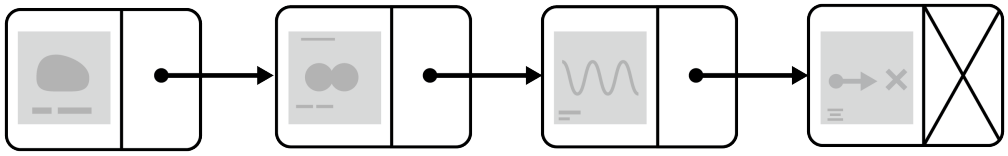
In this example, we first create three nodes using the ‘new‘ keyword, which dynamically allocates memory. We then assign data to the nodes and link them using the ’next’ pointer.

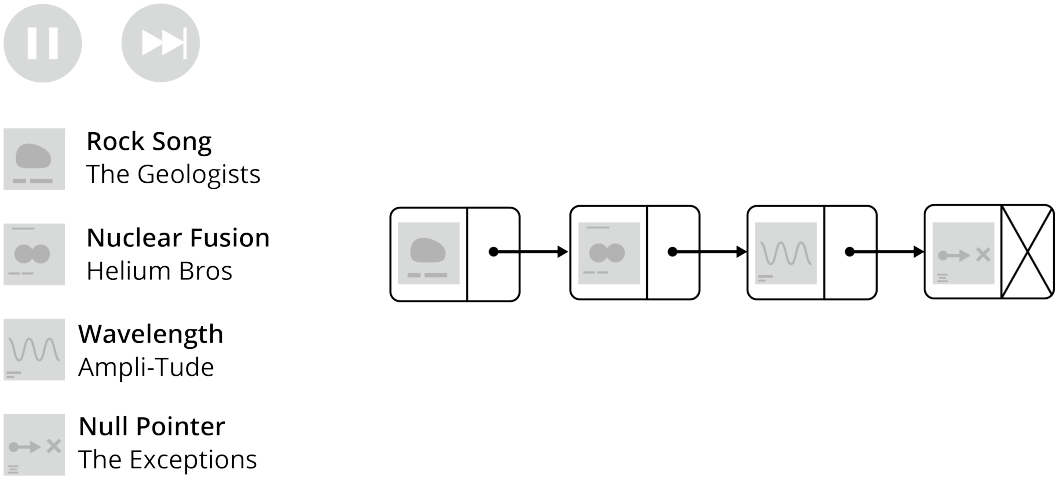
Rock Song
The Geologists

Nuclear Fusion
Helium Bros

Wavelength
Ampli-Tude

Null Pointer
The Exceptions

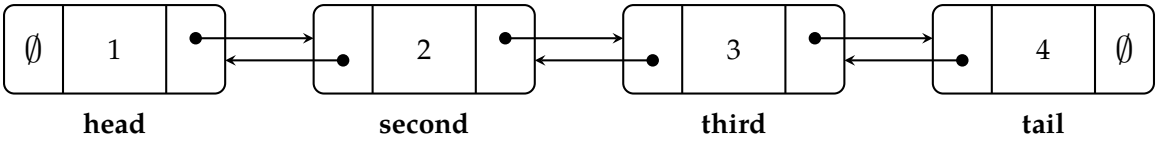



Rock Song
The Geologists
Nuclear Fusion
Helium Bros
Wavelength
Ampli-Tude
Null Pointer
The Exceptions

5.3 Doubly Linked Lists

Now that we have established the basics of linked lists, we can move on to a more complex structure: the *doubly linked list*. Doubly linked lists are strong because each node now has two pointers: the same next pointer, and now, the *predecessor*, which we will call `prev`.

```
struct DoublyNode {
    int data;
    DoublyNode* next;
    DoublyNode* prev;
};
```



Given this structure, we can traverse the entire list starting at the tail to the head or from the head to the tail.

Let's construct some code to analyze where the nodes would be in memory. We will create a doubly linked list with four nodes, and then we will print out the addresses of each node and their respective next and previous pointers.

Write the following code for the doubly linked list. There is a full file in your digital resources, here is a snippet of the code to get you started.

```

int main() {
    // Create nodes
    DoublyNode* head = new DoublyNode();
    DoublyNode* second = new DoublyNode();
    DoublyNode* third = new DoublyNode();
    DoublyNode* tail = new DoublyNode();

    // Assign data
    head->data = 1;
    second->data = 2;
    third->data = 3;
    tail->data = 4;

    // Link nodes forward
    head->next = second;
    second->next = third;
    third->next = tail;
```

```
    tail->next = nullptr;   // tail has no successor

    // Link nodes backward
    head->prev = nullptr;   // head has no predecessor
    second->prev = head;
    third->prev = second;
    tail->prev = third;

    // Print the address of each node, and the addresses stored in its
    // prev/next fields, so you can see both directions of the chain.
    std::cout << "head   is at: " << head   << "   prev: " << head->prev   << "
    ↪  (nullptr)" << "   next: " << head->next   << std::endl;
    std::cout << "second is at: " << second << "   prev: " << second->prev << "
    ↪  next: " << second->next << std::endl;
    std::cout << "third  is at: " << third  << "   prev: " << third->prev  << "
    ↪  next: " << third->next  << std::endl;
    std::cout << "tail   is at: " << tail   << "   prev: " << tail->prev   << "
    ↪  next: " << tail->next   << " (nullptr)" << std::endl;

    // Each DoublyNode takes up a fixed number of bytes in memory.
    std::cout << "\nsizeof(DoublyNode) = " << sizeof(DoublyNode) << " bytes" <<
    ↪  std::endl;

    // Clean up the memory
    delete head;
    delete second;
    delete third;
    delete tail;

    return 0;
}
```

When you run this, you'll get real addresses like these (yours specific addresses will differ!):

```
head   is at: 0x100c9dca0   prev: 0 (nullptr)   next: 0x100c9dbc0
second is at: 0x100c9dbc0   prev: 0x100c9dca0   next: 0x100c9dbe0
third  is at: 0x100c9dbe0   prev: 0x100c9dbc0   next: 0x100c9dc00
tail   is at: 0x100c9dc00   prev: 0x100c9dbe0   next: 0 (nullptr)

sizeof(DoublyNode) = 24 bytes
```

Computer memory is comprised of a very long string of bytes, and each byte has its own address. This line is a large, contiguous stream of ones and zeros when you get to the core of it. Often, programmers separate it into chunks of certain sizes so that we can make them look similar to grids of memory, like matrices.

Look closely at `second` and `third`: `0x100c9dbe0 - 0x100c9dbc0 = 0x20`, which is **32** in

decimal, not 24 bytes like `sizeof(DoublyNode)` had provided. The same is true between `third` and `tail`. Why, then, is the a difference between the 32 stored bytes and 24 bytes we would expect our memory to take up?

`DoublyNode` holds an `int` (4 bytes) plus two pointers (8 bytes each on a 64-bit machine), for 24 bytes total. But heap allocators don't hand out memory in exactly the size you ask for, they *round each allocation up* to a boundary of 16 bytes. Since 24 isn't a multiple of 16, the allocator rounds it up to the next one: 32. Your struct only uses 24 of those bytes; the other 8 are allocator *padding* that `sizeof()` never shows you. This is why the command new combines `malloc` and `sizeof`, and then proceeds to calculate the next multiple of 16 bytes to allocate.

Compare this to the `Node` struct from the previous section, which only has one pointer and one integer. It is 12 bytes, but memory allocation rounds it up to 16 bytes. This is why the addresses of the nodes in a linked list are not always exactly the size of the struct apart from each other. The addresses, too, may not always be consecutive.

Alternatively, comparing this to data structures in Python like Lists, we can see that Python's lists are more like arrays than linked lists. Python's lists are implemented as dynamic arrays, which means that they are contiguous blocks of memory that can grow and shrink in size. This allows for fast access to elements by index, but can be less efficient for inserting or deleting elements in the middle of the list, as it may require shifting elements around in memory. In contrast, linked lists allow for efficient insertion and deletion of elements at any position in the list, but require more memory overhead (like temporary variables) due to the additional pointers needed to link the nodes together.

Chapter 6

Bitmaps

Let's talk about another image format: the Bitmap. The Bitmap, denoted with file type `.bmp`, is an early file format for storing images on computers. Rows of different colors are stored as a grid of pixels. Each pixel is represented by a specific number of bits, which determines the color depth of the image. For example, a 24-bit bitmap can represent over 16 million colors, while an 8-bit bitmap can only represent 256 colors.

Bitmaps are uncompressed, meaning they can take up a lot of storage space compared to other image formats like JPEG or PNG. However, they are simple to read and write in code, making them useful for certain applications where image quality is more important than file size.

To understand how bitmaps work in practice, it helps to think of an image as nothing more than numbers stored in memory. Each pixel in a bitmap corresponds to a color value, and these values are usually written in hexadecimal (base-16) form. If you need a review, look back at Chapter **??**

In a common 24-bit bitmap, each pixel is made up of three color components:

- Red
- Green
- Blue

The intensity of each color component is represented by 8 bits. In decimal, this means each component can have a value from `0` to 255, for a total of 256 possible values. In hexadecimal, this range is represented from `00` to `FF`.

Each pixel's color is then represented by a 6-digit hexadecimal number, where the first two digits represent the red component, the next two represent green, and the last two represent blue. A total of $16,777,216$ different colors can be represented $(256 \times 256 \times 256)$.

Together, the intensity of each component determines the final color of the pixel. For example:

- `0xFF0000` represents pure red
- `0x00FF00` represents pure green
- `0x0000FF` represents pure blue

- `0xFFFFFF` represents white
- `0x000000` represents black

Recall that the prefix `0x` indicates that the number is in hexadecimal format. When creating or manipulating bitmap images in code, you can directly set the color values of individual pixels using their hexadecimal representations. This allows for precise control over the image's appearance.

6.1 Bitmap Structure

When traversing a bitmap, it's important to understand its structure. A bitmap file typically consists of a header followed by the pixel data. The header contains metadata about the image, such as its width, height, and color depth. The pixel data is stored in a grid format, with each pixel represented by its color value.

In a 24-bit bitmap, each pixel is represented by 3 bytes (one byte for each color component). The pixel data is usually stored in a bottom-up order, meaning the first row of pixel data corresponds to the bottom row of the image.

6.1.1 Bitmap Headers

The bitmap header is typically constructed with a file section and info section. Following those comes the pixel data, represented as bytes.

```
| File Header (?) | Info Header (usually 40 bytes) | Pixel Data (rowSize*height) |
```

```

struct BITMAPFILEHEADER
{
    WORD bfType; //specifies the file type
    DWORD bfSize; //specifies the size in bytes of the bitmap file
    WORD bfReserved1; //reserved; must be 0
    WORD bfReserved2; //reserved; must be 0
    DWORD bfOffBits; //species the offset in bytes from the bitmapfileheader to the bitmap bits
};
struct BITMAPINFOHEADER
{
    DWORD biSize; //specifies the number of bytes required by the struct
    LONG biWidth; //specifies width in pixels
    LONG biHeight; //species height in pixels
    WORD biPlanes; //specifies the number of color planes, must be 1
```

```
    WORD biBitCount; //specifies the number of bit per pixel
    DWORD biCompression;//specifies the type of compression
    DWORD biSizeImage; //size of image in bytes
    LONG biXPelsPerMeter; //number of pixels per meter in x axis
    LONG biYPelsPerMeter; //number of pixels per meter in y axis
    DWORD biClrUsed; //number of colors used by th ebitmap
    DWORD biClrImportant; //number of colors that are important
};
```

You can refer to the BMP file format specification and the bitmap article for more information. Let's create variables to hold important fields:

- `int w = bih.biWidth;` - width of the image in pixels
- `int h = bih.biHeight;` - height of the image in pixels
- `int size = bih.biSizeImage;` - the total size of the pixel data in bytes
- Note that we create `bytesPerPixel = 3` - the number of bytes per pixel (3 for 24-bit RGB)

Let's also preemptively build a pixel structure that can contain the information of an individual value at coordinate (x, y):

```
struct PIXEL
{
    // Each value is in range 0 to 255 represented as a byte
    BYTE b; // 1 byte
    BYTE g; // 1 byte
    BYTE r; // 1 byte
};
```

6.1.2 Padding

Remember that bytes are stored in a contiguous block of memory, as one long string of bytes. The computer does not inherently know where one row ends and the next begins (a concept called **row-major order**)[1]. We need to calculate the starting index of each pixel based on its row and column position. But, there is an issue: each row of pixel data in a bitmap must be aligned to a 4-byte boundary.

Padding exists in bitmap images to ensure that each row of pixel data is aligned to a 4-byte boundary in memory, which was a design choice made to improve performance and

[1] In row-major order, 2D arrays like bitmaps are stored sequentially in memory, row by row. To review this concept, refer back to the Vectors and Matrices Chapter, which explains how matrices can be analyzed in row-major order.

simplicity on early computer systems. Processors and hardware are more efficient when reading data that begins at predictable, aligned memory addresses, and forcing each row to occupy a size divisible by four bytes guarantees this alignment.

Because bitmap pixels do not always naturally fill a multiple of four bytes—especially in formats like 24-bit images where each pixel uses three bytes—extra, non-image bytes are added to the end of each row to reach the required alignment. These padding bytes do not represent color information and are ignored when displaying the image, but they ensure that each row starts at a consistent location in memory, making bitmap files easier and faster for software and hardware to process.

This means that if the width of the image (in bytes) is not a multiple of 4, we need to add padding bytes at the end of each row to ensure proper alignment. Padding, then, is extra bytes added to the end of each row of pixels so that the row size is a multiple of 4 bytes.

Let's look at an example. If we create a 4 pixel bitmap, it takes up

$$4 \text{ pixels} \times 3 \text{ bytes} = 12 \text{ bytes}$$

The memory looks something like

```
... | ?? | ?? | [B G R] [B G R] [B G R] [B G R] | ?? | ?? | ...
```

Since memory is contiguous, the question marks represent unrelated memory that does not belong to the bitmap's pixel data. Attempting to access memory outside the bounds of the image means the program is reading or writing data it does not own. Modern operating systems protect memory by dividing it into regions assigned to each program. If a program tries to access memory outside of its permitted region, the operating system immediately stops the program and reports an error known as a `Segmentation Fault`. This mechanism prevents programs from corrupting other data and helps ensure overall system stability.

12 bytes is evenly divisible by 4, so *no padding is needed*.

However, if we create a 3 pixel bitmap, it would take up:

$$3 \text{ pixels} \times 3 \text{ bytes} = 9 \text{ bytes}$$

9 is not evenly divisible by 4, so 3 bytes of memory must be added as padding for row-alignment.

```
... | ?? | ?? | [B G R] [B G R] [B G R] [ P ] [ P ] [ P ] | ?? | ?? | ...
```

Note that we read the pixels in `B G R` order in memory, but hex colors are read usually to

humans via `R G B` order. This gets into a concept called *little endian*. Put simply in a stack overflow post:

> RGB is a byte-order. But a deliberate implementation choice of most vanilla Graphics libraries is that they treat colours as unsigned 32-bit integers internally, with the three (or four, as alpha is typically included) components packed into the integer.
>
> On a little-endian machine (such as x86) the integer `0x01020304` will actually be stored in memory as `0x04030201`. And thus `0x00BBGGRR` will be stored as `0xRRGGBB00`!

So how can we create an equation for finding a pixel at the coordinates (x, y) if memory is in a contiguous line?

The calculation `rawRowSize = width * bytesPerPixel` provides a raw row estimate, but if padding is included, we need to round up by 4. The multiples of 4 look like 0, 4, 8, 12, 16, 20, 24, 28, 32..

- If `rawRowSize` is already one of these, we want to keep it.
- If not, we want to round upwards to the next multiple.

One way to accomplish this rounding is to take advantage of integer division. By adding a small offset before dividing, we can ensure that any value which is not already divisible by 4 is pushed into the next group of four bytes. Specifically, adding 3 to the raw row size guarantees that the result will round upward when divided by 4 using integer arithmetic.

This gives us the following expression for the true number of bytes in a single row, including padding:

$$\texttt{rowSize} = \left\lfloor \frac{\texttt{rawRowSize} + 3}{4} \right\rfloor \times 4$$

In C++, this is commonly written as:

```
1 int rowSize = ((width * bytesPerPixel + 3) / 4) * 4;
```

This value represents the number of bytes that must be traversed in memory to move from the beginning of one row of pixels to the beginning of the next. Recall that our `bytesPerPixel` is 3, but can also be calculated by `int bytesPerPixel = bih.biBitCount / 8;`.

Once the padded row size is known, we can compute the location of any pixel at coordinates (x, y). Because bitmap pixel data is stored row by row in a contiguous block of

memory, the offset to the start of row y is given by

$$y \times rowSize$$

Within that row, each pixel occupies `bytesPerPixel` bytes, so the offset to column x is:

$$x \times \texttt{bytesPerPixel}$$

Together, these two offsets combined yields the final memory index for pixel (x, y):

$$\texttt{index}(x, y) = y \times \texttt{rowSize} + x \times \texttt{bytesPerPixel}$$

In code, this calculation appears as:

```
int offset = y * rowSize + x * bytesPerPixel;
```

This formula will come in handy later in our code!

Let's write the full code for bitmap analysis. Starting with store the bitmap that our program reads into a memory allocated array, we can load our bitmap into a file, and read all the file information from it. We also can then utilize our equations `rowSize` and `idx` to get the individual pixel at $(x, y) \in (w, h)$.

```
// program arguments: ./bitmap_reader inputname.bmp outputname.bmp
if (argc !=3) return 1;
string inputname = argv[1];
string outputname = argv[2];


FILE *inputfile = fopen(inputname.c_str(), "rb"); // read byte only mode

BITMAPFILEHEADER bfh;
BITMAPINFOHEADER bih;

fread(&bfh.bfType, 2, 1, inputfile);
fread(&bfh.bfSize, 4, 1, inputfile);
fread(&bfh.bfReserved1, 2, 1, inputfile);
fread(&bfh.bfReserved2, 2, 1, inputfile);
fread(&bfh.bfOffBits, 4, 1, inputfile);
fread(&bih, sizeof(BITMAPINFOHEADER), 1, inputfile);

int w = bih.biWidth;
int h = bih.biHeight;
int bytesPerPixel = bih.biBitCount / 8; // 3 BYTES, stored in BGR order
int rowSize = ((w * bytesPerPixel + 3) / 4) * 4;
int size = rowSize * abs(h);
```

```
bih.biSizeImage = size;
bfh.bfSize = bfh.bfOffBits + bih.biSizeImage;

fseek(inputfile, bfh.bfOffBits, SEEK_SET); // offset from header
BYTE* data = (BYTE *)malloc(size);
fread(data, size, 1, inputfile);
fclose(inputfile);

// ----- FORCE STANDARD BMP HEADER HERE -----
bih.biSize = 40;
bih.biSizeImage = size;
bfh.bfOffBits = 54;
bfh.bfSize = 54 + size;
```

From there, we can set up an output file:

```
FILE *outfile = fopen(outputname.c_str(), "wb"); // creates a new file in write
↪  bytes mode
BYTE* out = (BYTE *) malloc(size);

fwrite(&bfh.bfType,     2, 1, outfile);
fwrite(&bfh.bfSize,     4, 1, outfile);
fwrite(&bfh.bfReserved1,2, 1, outfile);
fwrite(&bfh.bfReserved2,2, 1, outfile);
fwrite(&bfh.bfOffBits,  4, 1, outfile);
fwrite(&bih, sizeof(BITMAPINFOHEADER), 1, outfile);
```

And then, we can loop through to the `w` and `h` to analyze each individual pixel. What we will do as a first test of pixel alteration is *swap red and blue values* in our output.

```
for (int x = 0; x < w; x++)
{
    for (int y = 0; y < h; y++)
    {
        int idx = y * rowSize + x * 3;

        PIXEL p;
        BYTE B = data[idx];
        BYTE G = data[idx + 1];
        BYTE R = data[idx + 2];
        p = { B, G, R };

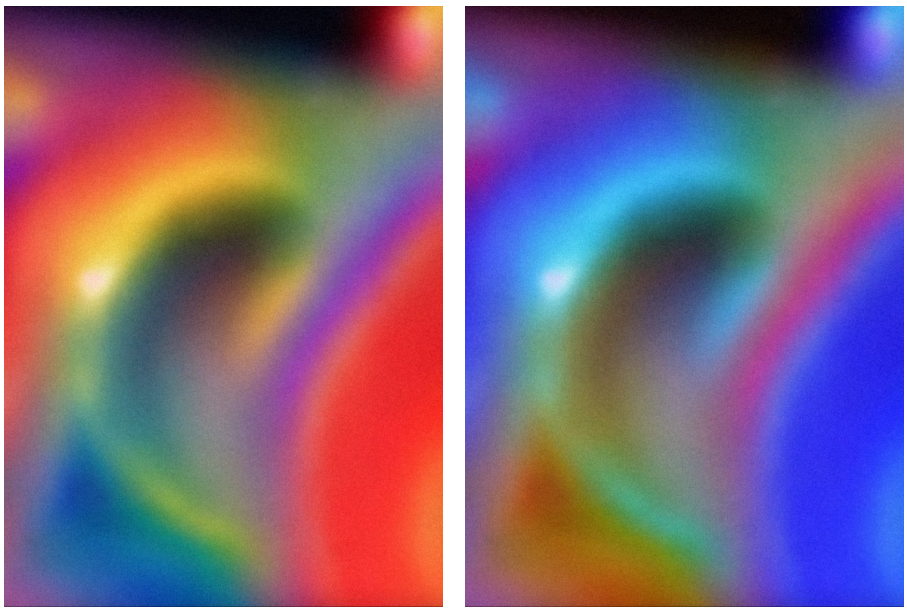
        PIXEL o;
        o = { R, G, B }; // Swaps red and blue values
        out[idx + 0] = o.b;
        out[idx + 1] = o.g;
        out[idx + 2] = o.r;
    }
```

```
19 }
```

Note that we search our data array at `idx` to locate the blue component, and adding 1 and 2 respectively gets the next two bytes (green and red). This works because

- Memory is a linear array of bytes
- Pixels are stored contiguously
- The order is fixed by the BMP format

After compiling, running this using `./bitmap_reader cow.bmp output.bmp` and `./bitmap_reader gradient.bmp output.bmp` produces the following outputs (see Figures **??** and **??**):



(a) Input bitmap of a gradient. (b) Output bitmap.

Figure 6.1: Running `bitmap_reader.cpp` on a gradient, swapping red and blue channels.



(a) Input image of a highland cow. (b) Output bitmap.

Figure 6.2: Running `bitmap_reader.cpp` on a gradient, swapping red and blue channels.

The entire base code for swapping the red and blue channels is in your Digital Resources, as well as a basic copy paste input-output bitmap. A lot of this stays the same of other pixel-based modifications, such as the next Exercise.

Exercise 5 Eliminate the Green component

Now try it yourself:

Alter both `cow.bmp` and `gradient.bmp` such that the green channel is eliminated. Keep the red and blue channels the same (don't swap them).

Working Space

Answer on Page 68

6.2 Creating a Bitmap with C++

What if we aren't given an input file? Can we create a bitmap?

Of course, let's create a simple 3x3 bitmap of the following colors:

```
[ Red ] [ White ] [ White ]
[ Black ] [ Blue ] [ White ]
[ Black ] [ Black ] [ Green ]
```

Although the image is conceptually two-dimensional, the bitmap file stores its pixel data as a one-dimensional sequence of bytes.

There are, however, a few constraints:

- 14-bit file header
- 40-byte info header
- pixel data with required row padding

```
| File Header (14) | Info Header (40) | Pixel Data |
```

Calculations:

- Each pixel $= 3$ `bytes`
- Row $= 3$ `pixels` $\times 3$ `bytes per pixel` $= 9$ `pixels`
- 3 `padding bytes` $\implies 12$ `bytes per row`
- 12 `bytes` $\times 3$ `rows` $= 36$ `bytes`

Here is the basic bitmap struct with filled in values:

```c
#pragma pack(push, 1)
struct BITMAPFILEHEADER {
    uint16_t bfType = 0x4D42; // 'BM'
    uint32_t bfSize;
    uint16_t bfReserved1 = 0;
    uint16_t bfReserved2 = 0;
    uint32_t bfOffBits = 54; // 14 + 40
};

struct BITMAPINFOHEADER {
    uint32_t biSize = 40;
    int32_t  biWidth = 3;
    int32_t  biHeight = 3;
    uint16_t biPlanes = 1;
    uint16_t biBitCount = 24;
    uint32_t biCompression = 0; // BI_RGB
    uint32_t biSizeImage;
```

```
    int32_t  biXPelsPerMeter = 0;
    int32_t  biYPelsPerMeter = 0;
    uint32_t biClrUsed = 0;
    uint32_t biClrImportant = 0;
};
#pragma pack(pop)
```

These structures match the on-disk layout of a bitmap header exactly. The `#pragma pack` directive ensures that no padding bytes are inserted by the compiler.

Recall that we can reuse our calculations from the first example to find rowsize and other variables, but this time, we define them instead of fetching them from the input:

```
const int width = 3;
const int height = 3;
const int bytesPerPixel = 3;
const int rowSize = ((width * bytesPerPixel + 3) / 4) * 4; // 12
const int imageSize = rowSize * height; // 36
```

Now we can create a new file and write the file headers individually:

```
FILE* f = fopen("custom_made.bmp", "wb");

BITMAPFILEHEADER bfh;
BITMAPINFOHEADER bih;

bih.biSizeImage = imageSize;
bfh.bfSize = bfh.bfOffBits + imageSize;

fwrite(&bfh, sizeof(bfh), 1, f);
fwrite(&bih, sizeof(bih), 1, f);
```

At this point, the file contains only metadata. No pixel values have been written yet.

Recall that bitmap pixel data is stored bottom-up, meaning the first row written corresponds to the bottom row of the image.

```
Memory order:
Row 2 (bottom)
Row 1
Row 0 (top)
```

Let's write the third row first:

```
// ROW 3 (bottom)
// Black      Black          Green
row[0] = 0;   row[1] = 0;   row[2] = 0;      // Black
row[3] = 0;   row[4] = 0;   row[5] = 0;      // Black
row[6] = 0;   row[7] = 255; row[8] = 0;      // Green
fwrite(row, rowSize, 1, f);
```

Each group of three bytes represents one pixel in BGR order. Any remaining bytes in the row serve as padding and are ignored when the image is displayed.

Row 2:

```
// ROW 2
// Black     Blue           White
row[0] = 0;   row[1] = 0;   row[2] = 0;      // Black
row[3] = 255; row[4] = 0;   row[5] = 0;      // Blue
row[6] = 255; row[7] = 255; row[8] = 255;    // White
fwrite(row, rowSize, 1, f);
```

Row 1:

```
// ROW 1 (top)
// Red        White          White
row[0] = 0;   row[1] = 0;   row[2] = 255;    // Red
row[3] = 255; row[4] = 255; row[5] = 255;    // White
row[6] = 255; row[7] = 255; row[8] = 255;    // White
fwrite(row, rowSize, 1, f);
```

And that's it! The compiler automatically converts the 255s to `0xFF`. Close the file and run the program, and you get an extremely small output. Enlarge the output and you get:

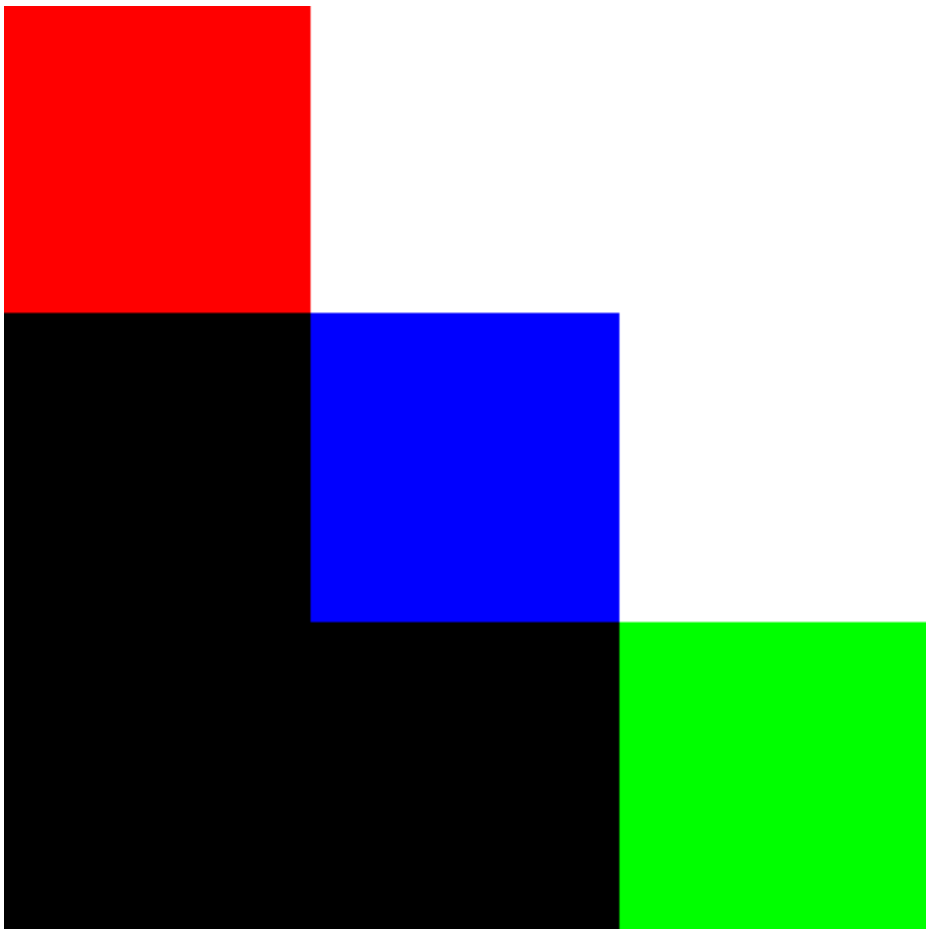


Figure 6.4: Your custom bitmap!

Exercise 6 8 Pixel Colorful Bitmap

Working Space

Create a 4×2 bitmap with the following layout:

```
[ Yellow ] [ Magenta ] [ Cyan ] [ White ]
[ Black  ] [ Red     ] [ Green ] [ Blue  ]
```

Bitmap Requirements:

- Width: 4 pixels
- Height: 2 pixels
- Color depth: 24-bit (RGB, stored as BGR)
- Compression: None (BI_RGB)
- Row padding: Rows must be aligned to 4-byte boundaries

Answer the following questions before writing any code:

1. How many bytes does each pixel use?
2. How many bytes are required for one row before padding?
3. How many padding bytes are required per row?
4. What is the total size of the pixel data?
5. What is the total file size?

Answer on Page 69

6.3 Connection to transformations

What if alter the pixel values in a more complex way? For example, instead of altering the colors themselves, we alter the pixel positions. This is what is called a transformation, and is a fundamental concept in computer graphics. A transformation is a mathematical operation that changes the position, size, or orientation of an object in a graphical space. In the context of bitmaps, transformations can be applied to the pixel data to achieve various effects, such as rotation, scaling, translation, and shearing.

What if you took a selfie, but then wanted to mirror it horizontally, to get a more representation of your face? This is a common transformation called a horizontal flip. To achieve this, you would iterate through each row of pixels in the bitmap and swap the pixels on the left side with the corresponding pixels on the right side. This can be done by calculating the index of each pixel and performing the necessary swaps. Note that nothing on the vertical direction of the image is changed, only the horizontal positions of the pixels are altered.

Since we are storing our pixel data as (x, y) coordinates, we can easily apply transformations by manipulating these coordinates. For an image with width w and height h, the horizontal flip transformation can be expressed mathematically as:

$$x' = w - 1 - x, \qquad y' = y$$

This operation does not change the pixel values themselves, but instead sets the new position of each pixel according to the transformation rule. The entire image transformation is therefore a systematic coordinate mapping applied across the matrix.

Without having to learn too much linear algebra, we can express this as a simple function: If we have a pixel coordinate expressed as a simple vector $\mathbf{p} = \begin{bmatrix} x \\ y \end{bmatrix}$, we can define a transformation function T that takes this pixel coordinate and maps it to a new coordinate $\mathbf{p}'$:

$$\mathbf{p}' = T\mathbf{p}$$

In this case, the transformation function T is a matrix that encodes the horizontal flip operation that obeys matrix multiplication. We have not talked about this yet, and it is a difficult concept to understand without having introduced linear algebra or matrices. So, we will not go into the details of how to construct this matrix, but just know that a horizontal flip can represented as:

$$T = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$$

This matrix, when multiplied with the pixel coordinate vector $\mathbf{p}$, will yield the new coordinate $\mathbf{p}'$ that corresponds to the horizontally flipped position of the original pixel.

Specifically, the -1 in the first row of the matrix indicates that the x-coordinate is inverted (flipped), while the 1 in the second row indicates that the y-coordinate remains unchanged.

Let's do this with code! The only thing that changes in our for loop is the calculation of the output pixel's position. Instead of writing to the same index, we write to the flipped index:

```
for (int y = 0; y < h; y++)
{
    for (int x = 0; x < w; x++)
    {
        int newX = w - 1 - x; // Calculate the new x-coordinate for the
        ↪ horizontal flip
        int srcIdx = y * rowSize + x * 3;
        int dstIdx = y * rowSize + newX * 3;

        // read the pixel values from the source index
        BYTE B = data[srcIdx];
        BYTE G = data[srcIdx + 1];
        BYTE R = data[srcIdx + 2];

        //output the pixel values to the destination index
        out[dstIdx + 0] = B;
        out[dstIdx + 1] = G;
        out[dstIdx + 2] = R;
    }
}
```

Taking the input bitmap of the cow from 6.2a and running this code produces the horizontally flipped output in 6.6.



Figure 6.6: Output of a horizontally flipped cow bitmap after applying the new pixel position transformation.

Similarly, we can transform the bitmap in other ways, which we will learn more of how to do in the transformations chapter by experimenting with vectors. For now, remember that matrices are a powerful tool for representing transformations, and that by manipulating pixel coordinates, we can achieve a wide variety of effects on bitmap images.

6.4 Summary

In this chapter, we experimented with bitmaps. Specifically,

- loading a bitmap in C++

- creating and copying bitmap file and info headers
- creating a pixel structure to work with hexadecimal bytes and hexcodes
- swapping the channels of a bitmap
- creating a bitmap from scratch in C++

Appendix A

Answers to Exercises

Answer to Exercise 1 (on page 6)

The first vector of the orthogonal subspace is:

$$v_1 = x_1 = (1, 1, 1)$$

The second vector of the subspace is a projection of x_2 onto v_1.

$$v_2 = x_2 - \frac{x_2 v_1}{v_1 v_1} v_1$$

Substitute the values:

$$v_2 = (0, 1, 1) - \frac{(0, 1, 1)(1, 1, 1)}{(1, 1, 1)(1, 1, -1)}(1, 1, 1)$$

$$v_2 = (0, 1, 1) - (2/3)(1, 1, 1)$$

$$v_2 = (-2/3, 1/3, 1/3)$$

Normalize:

$$v_1 = v_1 / \sqrt{|v_1|}$$

$$v_1 = (1, 1, 1) / \sqrt{|v_1|}$$

$$v_1 = (0.577, 0.577, 0.577)$$

$$v_2 = v_2 / \sqrt{|v_2|}$$

$$v_2 = (0, 1, 1) \sqrt{|v_2|}$$

$$v_2 = (-0.816, 0.408, 0.408)$$

Answer to Exercise 2 (on page 14)

$$U = \begin{bmatrix} -0.70710678 & -0.70710678 \\ -0.70710678 & 0.70710678 \end{bmatrix}$$

$$Singular values = [3.464101623.16227766]$$

$$V^T = \begin{bmatrix} -0.408 & -0.816 & -0.408 \\ -0.894 & 0.447 & 0.0 \\ -0.183 & -0.365 & 0.9129 \end{bmatrix}$$

Answer to Exercise 3 (on page 21)

Yes. Its eigenvalues are

$$2, 2$$

Answer to Exercise 4 (on page 21)

The answer depends on the 3 by 3 matrix you chose.

Answer to Exercise 5 (on page 57)

The only thing that changes in our for loop is the writing of the output pixel.

```
// CODE ABOVE STAYS THE SAME
for (int x = 0; x < w; x++)
{
    for (int y = 0; y < h; y++)
    {
        int idx = y * rowSize + x * 3;

        PIXEL p;
        BYTE B = data[idx];
        BYTE G = data[idx + 1];
        BYTE R = data[idx + 2];
        p = { B, G, R };

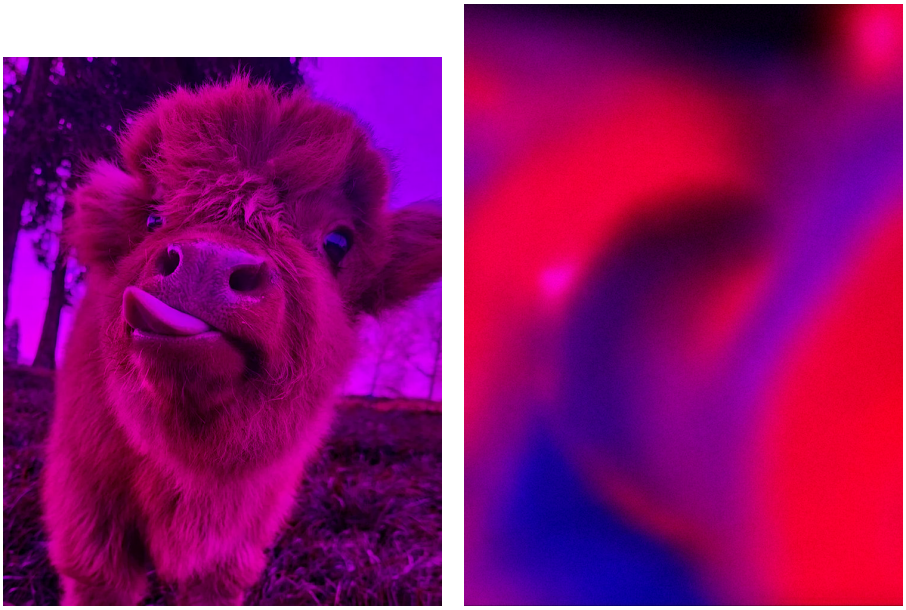
        out[idx + 0] = p.b;
        out[idx + 1] = 0; // NOTE THE CHANGE HERE
        out[idx + 2] = p.r;
    }
}
```

What is happening here?

In an RGB image, each pixel's final color is the combination of red, green, and blue intensities. If the green value is removed, every pixel changes from (R,G,B) to (R,0,B). Colors that relied heavily on green—such as greens, yellows, and many skin tones—lose a ma-

jor part of their intensity and appear much darker or shifted in hue. Pure green areas become black, yellow areas (red + green) become red, and cyan areas (green + blue) become blue.

Visually, the image often takes on a magenta or purplish tint (see Figures 6.3a and 6.3b), because magenta is the combination of red and blue with no green. Overall brightness decreases, since green contributes significantly to the brightness in human vision. Refer to Figures 6.2a and 6.1a for the original bitmaps.



(a) Output bitmap - cow. (b) Output bitmap - gradient.

Figure 6.3: Running `bitmap_no_green.cpp` on both provided bitmaps.

Answer to Exercise 6 (on page 62)

1. How many bytes does each pixel use? - 3 `bytes per pixel`
2. How many bytes are required for one row before padding? 4 `pixels`$\times$3 `bytes per pixel` = 12 `bytes`
3. How many padding bytes are required per row? 0 `padding bytes`
4. What is the total size of the pixel data? 12 bytes per row $\times$ 2 `rows` = 24 `bytes`
5. What is the total file size, including header structs? 14 `bytes for BFH`+40 `bytes for BIH` = 54 `bytes` $\implies$ $24 + 54 = 78$ `bytes`

Remember that we need to write the rows bottom-up, so the second row is written first

followed by the first row.

Let's establish the colors needed. Remember that we need to swap to BGR order:

- Black = (0, 0, 0)
- Red = (0, 0, 255)
- Green = (0, 255, 0)
- Blue = (255, 0, 0)
- Yellow = (0, 255, 255) (R=255,G=255,B=0 $\implies$ BGR = 0,255,255)
- Magenta = (255, 0, 255) (R=255,B=255 $\implies$ BGR = 255,0,255)
- Cyan = (255, 255, 0) (G=255,B=255 $\implies$ BGR = 255,255,0)
- White = (255, 255, 255)

Overall the program looks like this:

```
#include <cstdio>
#include <cstdint>
#include <cstring>


#pragma pack(push, 1)
struct BITMAPFILEHEADER {
    uint16_t bfType = 0x4D42; // 'BM'
    uint32_t bfSize;
    uint16_t bfReserved1 = 0;
    uint16_t bfReserved2 = 0;
    uint32_t bfOffBits = 54;
};

struct BITMAPINFOHEADER {
    uint32_t biSize = 40;
    int32_t  biWidth = 4;
    int32_t  biHeight = 2;
    uint16_t biPlanes = 1;
    uint16_t biBitCount = 24;
    uint32_t biCompression = 0;
    uint32_t biSizeImage;
    int32_t  biXPelsPerMeter = 0;
    int32_t  biYPelsPerMeter = 0;
    uint32_t biClrUsed = 0;
    uint32_t biClrImportant = 0;
};
#pragma pack(pop)

```

```
30 int main(int argc, char const *argv[])
31 {
32     FILE* f = fopen("fourbytwo.bmp", "wb");
33 
34     BITMAPFILEHEADER bfh;
35     BITMAPINFOHEADER bih;
36 
37     const int width = 4;
38     const int height = 2;
39     const int bytesPerPixel = 3;
40     const int rowSize = ((width * bytesPerPixel + 3) / 4) * 4; // 12
41     const int imageSize = rowSize * height;                    // 24
42 
43     bih.biSizeImage = imageSize;
44     bfh.bfSize = bfh.bfOffBits + imageSize;
45 
46     fwrite(&bfh, sizeof(bfh), 1, f);
47     fwrite(&bih, sizeof(bih), 1, f);
48     uint8_t row[rowSize];
49 
50     // ----------------------------
51     // Write bottom row first:
52     // [ Black ] [ Red ] [ Green ] [ Blue ]
53     // ----------------------------
54     std::memset(row, 0, rowSize);
55 
56     // Black
57     row[0] = 0;   row[1] = 0;   row[2] = 0;
58     // Red (BGR = 0,0,255)
59     row[3] = 0;   row[4] = 0;   row[5] = 255;
60     // Green (BGR = 0,255,0)
61     row[6] = 0;   row[7] = 255; row[8] = 0;
62     // Blue (BGR = 255,0,0)
63     row[9] = 255; row[10]= 0;   row[11]= 0;
64 
65     fwrite(row, rowSize, 1, f);
66 
67     // ----------------------------
68     // Write top row:
69     // [ Yellow ] [ Magenta ] [ Cyan ] [ White ]
70     // ----------------------------
71     std::memset(row, 0, rowSize);
72 
73     // Yellow (BGR = 0,255,255)
74     row[0] = 0;   row[1] = 255; row[2] = 255;
75     // Magenta (BGR = 255,0,255)
76     row[3] = 255; row[4] = 0;   row[5] = 255;
77     // Cyan (BGR = 255,255,0)
78     row[6] = 255; row[7] = 255; row[8] = 0;
79     // White (BGR = 255,255,255)
80     row[9] = 255; row[10]= 255; row[11]= 255;
81 
```

```
82      fwrite(row, rowSize, 1, f);
83
84      fclose(f);
85      return 0;
86  }
```

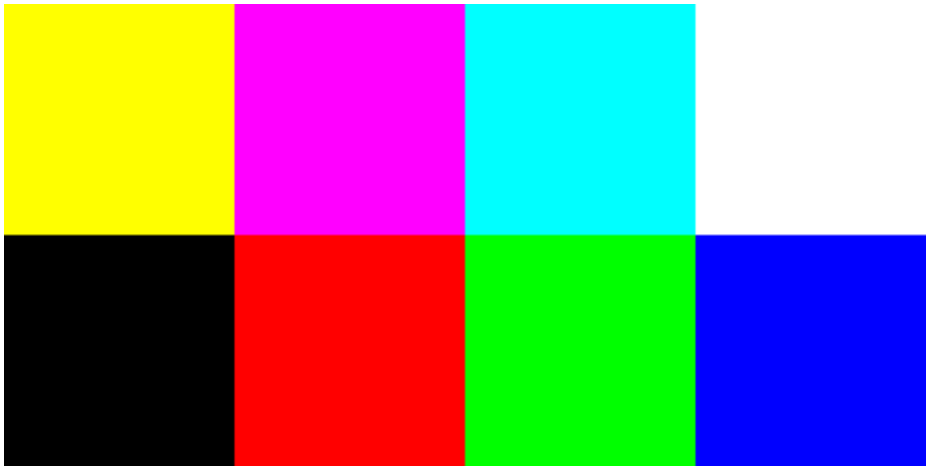
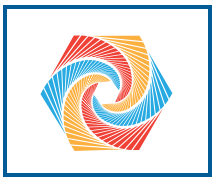


Figure 6.5: Exercise Output of a 4 by 2 bitmap.



INDEX

address-of operator, 31
arrow operator, 36

bitmaps, 49
    structure of, 50

C types, 27
classes, 41
compiler, 25

dereferencing, 32
doubly linked list, 46

free, 36

Gram-Schmidt Process, 3
Gram-Schmidt process
    python script for, 6

head, 41
heap, 36
    arrays, 38
hexadecimal, 49

linked list, 41

malloc, 36
Max Cut, 21
    python script for , 23
memory leak, 37
modf, 33

new, 41
node, 41
NULL, 33
null pointer, 42

padding, 48, 52
pass-by-reference, 33
pass-by-value, 31
pointer, 31, 32
predecessor, 46

segmentation fault, 52
singular value decomposition, 9
stack, 30
stack frame, 30
static typing, 27
struct, 35
successor, 41
svd, 9
    python script for, 13

tail, 41